hashlock.

Security Audit

Zeus Exchange (DeFi)

August 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

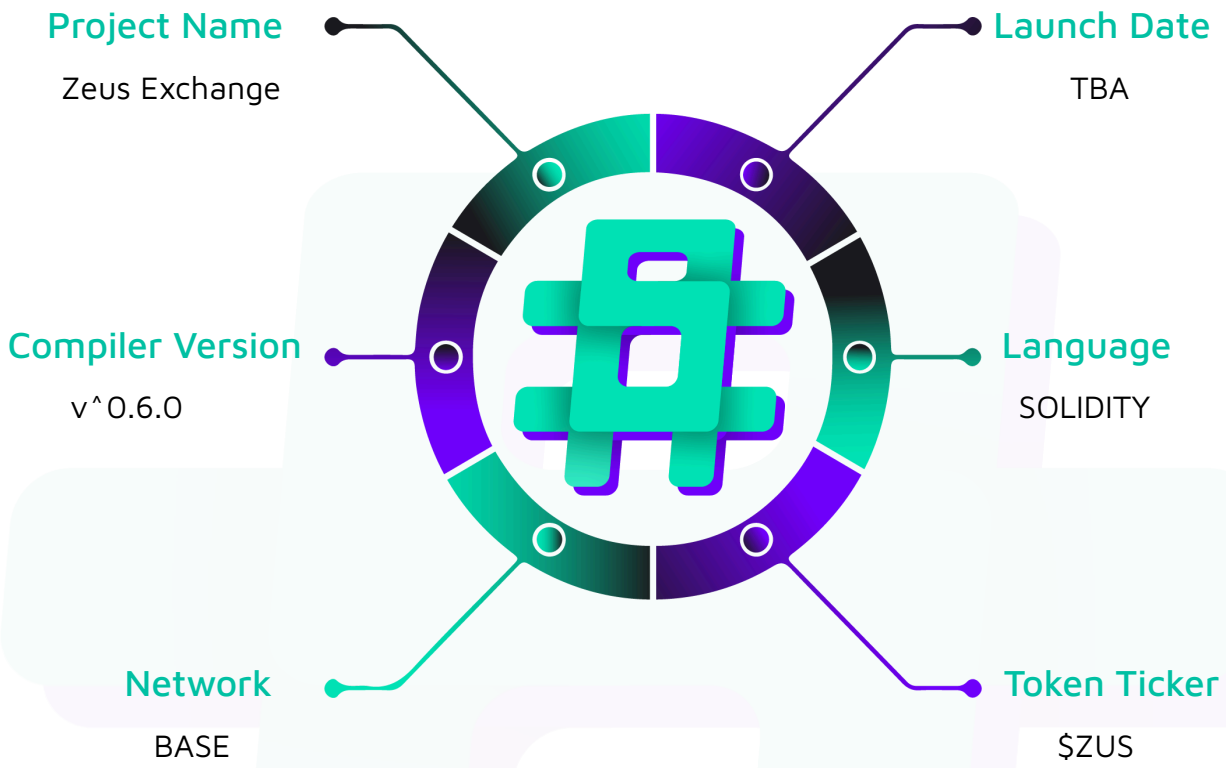
Zeus Exchange partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

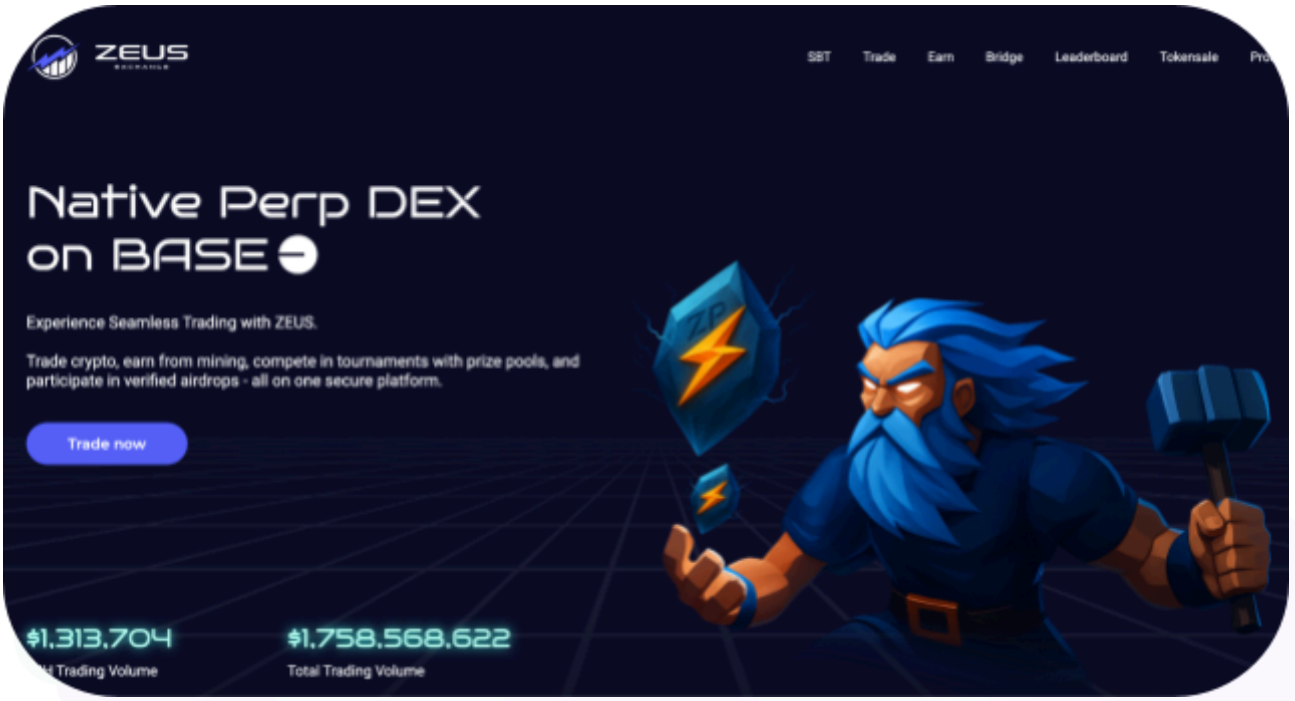
Zeus Exchange is a decentralized perpetual exchange (DEX) built on the Base blockchain. It enables both spot and perpetual (derivatives) trading directly from users' wallets without intermediaries. The platform sets itself apart by leveraging Soulbound Tokens (SBTs) to identify real users and fairly distribute 25% of monthly revenue as airdrops—combating airdrop abuse and promoting long-term engagement. With plans for multichain support, copy trading, AI agents, and RWA trading pairs, Zeus is positioning itself as a performance-driven, community-aligned player in DeFi's derivatives space.

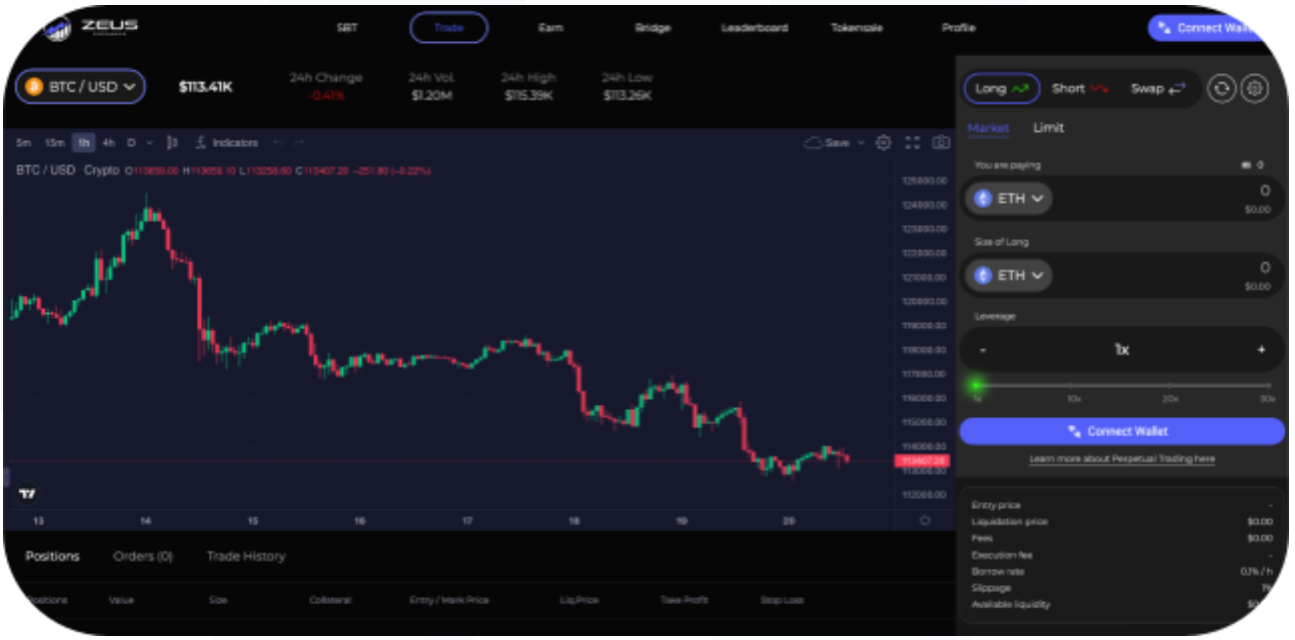
Project Name: Zeus Exchange
Project Type: DeFi
Compiler Version: ^0.6.0
Website: https://zeustrade.io/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the Zeus Exchange, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Zeus Exchange Smart Contracts**
Platform	**BASE / Solidity**
Audit Date	**August, 2025**
Contract 1	BasePositionManager.sol
Contract 2	OrderBook.sol
Contract 3	PositionManager.sol
Contract 4	PositionRouter.sol
Contract 5	PositionUtils.sol
Contract 6	Router.sol
Contract 7	ShortsTracker.sol
Contract 8	Vault.sol
Contract 9	VaultErrorController.sol
Contract 10	VaultPriceFeed.sol
Contract 11	VaultUtils.sol
Contract 12	ZlpManager.sol
Audited GitHub Commit Hash	fc0f6241b08603d8838e94ebb2b8559061a50c41
Fix Review GitHub Commit Hash	593ce587d9032919d24860b02ccdd6ed1ba880e2

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

3 High severity vulnerabilities

4 Medium severity vulnerabilities

4 Low severity vulnerabilities

1 Gas Optimisation

3 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
BasePositionManger.sol Allows for (user-initiated actions): - Increasing the size of leveraged long or short positions. - Decreasing or closing out leveraged long or short positions. - Registering position changes with the referral system to track rewards. Allows admins and governance to: - Withdraw protocol fees collected in various tokens. - Set the maximum total size for long and short positions for each asset. - Set and adjust the deposit fee percentage. - Update key contract addresses like admin and referralStorage. - Approve token spending by other contracts or directly send ETH. - Configure operational parameters like the gas limit for ETH transfers.	**Contract achieves this functionality.**
OrderBook.sol Allows users to: - Create trigger-based orders for swaps and leveraged positions (e.g., limit, stop-loss, take-profit). - Place conditional swap orders that execute when a target price ratio between two tokens is met. - Place orders to automatically increase a	**Contract achieves this functionality.**

leveraged position when an asset reaches a specific price. - Place orders to automatically decrease a leveraged position when an asset reaches a specific price. - Update the trigger conditions and amounts for their own pending orders. - Cancel any of their own pending orders and reclaim the deposited assets and execution fee. Allows anyone (as an executor/keeper) to: - Execute a pending order on behalf of a user once its price conditions are valid. - Receive an execution fee (paid in ETH by the order's creator) for successfully executing an order. Allows governance to: - Set the minimum execution fee required for creating any new order. - Set the minimum collateral value (in USD) for creating new position orders. - Initialize and update critical contract addresses (e.g., Router, Vault). - Change the contract's gov address.	
PositionManager.sol This contract extends BasePositionManager to serve as the primary entry point for managing leveraged positions, introducing role-based access control for different functions. Allows approved partners (on behalf of users) to: - Open, add to, or close leveraged long and short positions. - Use either ETH or ERC20 tokens as collateral for	**Contract achieves this functionality.**

opening positions. - Withdraw collateral in ETH or ERC20 tokens when closing positions. - Close a position and atomically swap the returned collateral into a different token in a single transaction. Allows whitelisted liquidators to: - Liquidate user positions that have fallen below the required collateralization ratio, securing the protocol. Allows whitelisted order keepers to: - Trigger the execution of pending swap, increase, and decrease orders that were created in the OrderBook contract. Allows admins/governance to: - Manage whitelists by granting or revoking permissions for partners, liquidators, and order keepers. - Enable or disable "legacy mode", which removes the partner whitelist requirement for managing positions. - Manage all protocol parameters inherited from BasePositionManager, such as fees, max global position sizes, and referral settings. - Toggle specific validation rules, like the check to prevent leverage reduction on new long positions.	
PositionRouter.sol This contract introduces an asynchronous request queue for managing leveraged positions. Instead of executing trades instantly, users create requests that are later processed by whitelisted keepers. This model	**Contract achieves this functionality.**

helps prevent front-running and allows for more reliable transaction execution. Allows users to: - Submit requests to open, increase, or decrease leveraged positions using either ETH or ERC20 tokens. - Pay an execution fee upfront, which is used to incentivize keepers to process their request. - Define an acceptable price for their trade to protect against slippage. - Manually execute or cancel their own requests after a public waiting period has passed. - Optionally register a callback contract to be notified when their request is fulfilled. Allows whitelisted position keepers to: - Execute or cancel queued user requests in batches, earning the associated execution fees. - Process requests after a shorter, privileged delay, ensuring the queue is handled efficiently. - Handle requests that may have failed due to issues like slippage or expiry by cancelling them. Allows admins/governance to: - Manage the whitelist of trusted position keepers. - Configure the minimum execution fee required to create a request. - Set the time and block delays that govern when keepers and users can execute requests. - Define the maximum time a request can remain pending before it expires. - Manage gas limits for external callbacks and access all administrative functions inherited from BasePositionManager (e.g., setting deposit fees,	

max position sizes, and withdrawing protocol revenue).	
PositionUtils.sol This is a library contract, which means it doesn't hold state or get deployed on its own. Instead, it provides reusable code and common logic that other position management contracts can use. Provides utility functions to: - Determine if a deposit fee should be charged. - Encapsulate the core logic for increasing a position.	**Contract achieves this functionality.**
Router.sol This contract is the primary entry point for all user interactions with the protocol, acting as a "router" that directs user funds and instructions to the core Vault for execution. It handles swaps, leveraged trading, and managing permissions for other contracts. Allows users to: - Swap Assets. - Manage Leveraged Positions. - Manage Plugin Permissions. Allows approved "plugin" contracts to: - Act on behalf of users who have granted them permission to perform actions like transferring tokens or opening/closing positions. This enables more complex trading logic (e.g., limit orders, position management bots). Allows governance to: - Whitelist new smart contracts as official	**Contract achieves this functionality.**

system-wide plugins. - Remove existing contracts from the plugin whitelist. - Update the governance address for the contract.	
ShortsTracker.sol This contract is a critical accounting and risk management component that monitors the aggregate state of all short positions within the protocol. It doesn't track individual user positions but rather the collective "global shorts" for each asset. Its primary functions are to: - Calculate Global Average Prices. - Track Global PnL. - Provide Data for Risk Management. Allows whitelisted "handler" contracts to: - Call the updateGlobalShortData function whenever a user's short position is modified (increased or decreased). This ensures the tracker's data is always in sync with the actual positions in the Vault. Allows governance to: - Manage Handlers. - Initialize Data. - Activate the Tracker.	**Contract achieves this functionality.**
Vault.sol This contract is the heart and brain of the entire protocol. It acts as a central bank, clearinghouse, and ledger, securely holding all protocol liquidity, managing every user position, and executing the core logic for all financial operations. Users typically interact with the Vault indirectly through a trusted Router.	**Contract achieves this functionality.**

Core Functionalities: - Unified Liquidity Pool. - Leveraged Trading Engine. - Synthetic Swaps & USDG Minting. - Accounting and Risk Management. Allows whitelisted managers & liquidators to: - Perform sensitive actions like minting/burning USDG when the protocol is placed in a protected "manager mode". - Liquidate undercollateralized positions to protect the protocol from bad debt, especially during high market volatility. Allows governance to: - Manage Assets. - Set All Fees. - Configure Risk Parameters. - Manage Roles & System Toggles - Withdraw Protocol Revenue.	
VaultErrorController.sol This is a simple, administrative helper contract designed to manage the human-readable error messages for the main Vault contract. Allows governance to: - Set Error Messages.	**Contract achieves this functionality.**
VaultPriceFeed.sol This contract is the dedicated and highly configurable price oracle for the entire protocol. Its sole purpose is to aggregate data from multiple sources to provide the Vault with a single, reliable, and manipulation-resistant	**Contract achieves this functionality.**

price for each asset. It acts as a sophisticated filter and validator that sits between raw Oracle data and the core Vault. Core Functionalities - Multi-Source Price Aggregation incorporates numerous safety checks to produce a trustworthy price. Allows governance to: - Configure All Sources. - Manage Token Configurations. - Tune All Risk Parameters. - Toggle Features.	
VaultUtils.sol This is a helper contract that offloads non-critical view functions, complex calculations, and position tracking logic from the main Vault contract. This architectural pattern helps keep the core Vault smaller, more secure, and cheaper to interact with by separating its core transactional logic from more complex, read-only computations. Allows the Vault contract to: - Register a new position in the global list when it's opened. - De-register a position from the global list when it's closed. Allows anyone (e.g., users, front-ends, bots) to: - Read the list of all open positions in the protocol. - Calculate and preview potential fees for any action (like a swap or opening a trade) before execution. - Check the liquidation status of any position	**Contract achieves this functionality.**

without needing to send a state-changing transaction.	
ZlpManager.sol This contract manages the protocol's liquidity provider (LP) token, ZLP. It serves as the primary interface for users to add and remove liquidity from the Vault. Allows whitelisted "handler" contracts to: - Add or remove liquidity on behalf of other user accounts. This enables integration with other DeFi protocols, such as yield aggregators or structured product vaults, that build on top of this system. Allows governance to: - Set Cooldown. - Adjust AUM. - Manage Handlers. - Configure Oracles.	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Zeus Exchange, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices, and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Zeus Exchange smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] VaultPriceFeed.sol::getPriceV2 - Spreads Are Not Applied in getPriceV2()

Description

The `getPriceV2` function is designed to calculate a price and then apply a configurable spread (a small percentage added or subtracted). However, a `return price;` statement is placed before the spread is calculated. This causes the function to return the raw price, making the entire spread mechanism unreachable.

Vulnerability Details

The premature return statement is located after the price is fetched but before the spread logic.

```
function getPriceV2(address _token, bool _maximise, bool _includeAmmPrice) public view
returns (uint256) {
        uint256 price;

        if (strictStableTokens[_token]) {

            return ONE_USD;
        }

        if (_includeAmmPrice && isAmmEnabled) {
            price = getAmmPriceV2(_token, _maximise, price);
        }

        if (isSecondaryPriceEnabled) {
            price = getSecondaryPrice(_token, price, _maximise);
        } else {
            price = getPrimaryPrice(_token, _maximise);
        }
```

```
        return price; //@audit: The function exits here, before applying the spread.

REDACTED
```

Impact

Spreads are a critical economic component for many DeFi protocols, serving as a safety buffer and a source of revenue. By failing to apply the spread, the oracle returns an incorrect price. Protocols using this oracle for V2 pricing would operate with no safety margin on their trades, potentially leading to losses and making the system financially unsustainable.

Recommendation

Remove the premature `return price;` statement. This will allow the program flow to continue to the spread calculation logic, ensuring the final price returned is correct.

Status

Resolved

[H-02] OrderBook.sol - Missing Access Control on cancelIncreaseOrderAdmin() and cancelDecreaseOrderAdmin() Orders Cancellation Admin Functions

Description

The smart contract contains two administrative functions, `cancelIncreaseOrderAdmin` and `cancelDecreaseOrderAdmin`. These functions are intended to allow a privileged user (like a governor or admin) to cancel any pending increase or decrease order on behalf of any user. However, these functions were declared with public visibility without an access control modifier to restrict who can call them.

Vulnerability Details

Because the functions lack any access control, any external account can call them. An attacker can simply call `cancelIncreaseOrderAdmin` or `cancelDecreaseOrderAdmin`, passing in the `_address` of a victim and the `_orderIndex` of one of their pending orders. The contract will process this request without verifying the caller's identity, assuming it's a legitimate administrative action.

Affected Code Snippets:

```
//@audit: Anyone can call this function for any user (_address)
function cancelIncreaseOrderAdmin(address _address, uint256 _orderIndex) public
nonReentrant {
        IncreaseOrder memory order = increaseOrders[_address][_orderIndex];
        require(order.account != address(0), "OrderBook: non-existent order");
    // ... logic to cancel the order and refund assets
}

//@audit: Anyone can call this function for any user (_address)
function cancelDecreaseOrderAdmin(address _address, uint256 _orderIndex) public
nonReentrant {
        DecreaseOrder memory order = decreaseOrders[_address][_orderIndex];
        require(order.account != address(0), "OrderBook: non-existent order");
    // ... logic to cancel the order and refund the fee
}
```

Impact

It enables a griefing attack that results in a functional denial of service across the entire protocol. Malicious actors can undermine protocol reliability by monitoring the blockchain for newly created orders and immediately canceling them, preventing users from ever having their orders filled and rendering the platform's core functionality unusable and untrustworthy. The vulnerability also opens the door to economic attacks, where attackers could selectively cancel orders to manipulate market conditions or obstruct specific users from entering or exiting positions at advantageous times.

Recommendation

Enforce proper access control. The existing `onlyGov` modifier should be added to both function definitions. This will restrict their execution to the designated `gov` address, ensuring that only the authorized administrator can cancel orders on behalf of users.

Status

Resolved

[H-03] OrderBook.sol - Reentrancy Attack in executeDecreaseOrder function allow attackers to inflate ZLP price and drain the vault

Description

Zeus Exchange contracts are forked from GMXv1 contracts.

In July 2025, GMXv1 contracts are exploited. For more details and information, please visit: https://x.com/GMX_IO/status/1943336664102756471

https://www.certik.com/resources/blog/gmx-incident-analysis

Vulnerability Details

The entry point of the attack was https://github.com/zeustrade/zeus-contracts/blob/main/contracts/exchange/core/OrderBook.sol#L977, The attacker exploited a reentrancy vulnerability, entering the `increasePosition` function unexpectedly via `OrderBook.sol` (line 977). Despite the presence of a `nonReentrant` modifier, this protection only applies within the `OrderBook` contract and doesn't guard calls to external contracts.

By exploiting this gap, the attacker triggered `increasePosition` in the `Vault` contract directly, bypassing the legitimate control flow (which otherwise would require routing through `PositionRouter` and `PositionManager`).

This manipulation caused the GMX V1 contract to miscalculate the average short price, which was artificially lowered from approximately $109,505.77 to $1,913.70.

Subsequently, the attacker executed a flash loan to purchase GLP at a low price (~$1.45), opened a large leveraged position. Due to the mispriced average short value, the position was massively profitable. After this, the attacker sold GLP when its price surged above $27, realizing a substantial gain.

Impact

The exploit resulted in around $40 million being drained from the GMX V1 deployment on Arbitrum.

Recommendation

- Disabling leverage via `Vault.setIsLeverageEnabled(false)` or `Timelock.setShouldToggleIsLeverageEnabled(false)`.
- Set all `maxUsdgAmounts` to "1" by using `Vault.` or `Timelock.setTokenConfig`, to prevent GLP minting. Note that the value must be "1" and not "0", as "0" would mean there is no cap.

Status

Resolved

Medium

[M-01] VaultPriceFeed.sol::getPrimaryPrice - Does Not Use the Primary Price Feed

Description

The `getPrimaryPrice` function is intended to provide a secure price from a primary Chainlink oracle by sampling several recent price rounds. However, the function's very first line of code is `return getSecondaryPrice(...)`, causing it to immediately exit and return a value from the secondary price feed instead.

Consequently, all the code designed to interact with the Chainlink oracle, including checking the Arbitrum sequencer status and sampling prices for robustness, is unreachable dead code. The function's name is completely misleading.

Vulnerability Details

The issue is caused by the misplaced return statement on the second line of the function body:

```
 function getPrimaryPrice(address _token, bool _maximise) public override view returns (uint256) {
         return getSecondaryPrice(_token, 0, _maximise); //@audit: This line causes the function to exit immediately.
       address priceFeedAddress = priceFeeds[_token];

REDACTED
```

Impact

Any system relying on this function assumes it's receiving a highly reliable and manipulation-resistant price from Chainlink. Instead, it receives a price from a secondary source, which may have different (and potentially lower) security guarantees. An attacker who finds a way to manipulate the secondary price feed could exploit this vulnerability to report false prices, leading to unfair liquidations, protocol insolvency, or theft of funds.

Recommendation

Remove the incorrect return statement to restore the function's intended logic. This will re-enable the crucial security feature of sampling multiple prices from the primary Chainlink feed.

Status

Resolved

[M-02] ZlpManager.sol - Cooldown Duration Can Be Bypassed

Description

The `ZlpManager` contract includes a cooldown mechanism intended to prevent users from withdrawing liquidity immediately after depositing it. When a user removes liquidity, the `_removeLiquidity` function checks if a specific amount of time, defined by `cooldownDuration`, has passed since their last deposit. This is implemented using a mapping, `lastAddedAt`, which stores a timestamp for the user's address (`_account`) every time they add liquidity.

The check is performed with this line:

```
require(lastAddedAt[_account].add(cooldownDuration) <= block.timestamp, "ZlpManager: cooldown duration not yet passed");
```

This security measure is designed to discourage short-term, speculative liquidity provisioning or to mitigate certain types of flash loan-based attacks where liquidity is added and removed within the same transaction.

Vulnerability Details

The vulnerability stems from the fact that the cooldown is tied to the user's address (`_account`), not the ZLP tokens themselves. Since ZLP is an ERC20 token, it can be freely transferred between different wallet addresses.

A user can easily circumvent the cooldown with a simple transfer:

1. Alice calls `addLiquidity` to deposit funds and receives ZLP tokens. Her address, Alice's Address, now has its `lastAddedAt` timestamp updated to the current `block.timestamp`.
2. If Alice tries to call `removeLiquidity` immediately from Alice's Address, the transaction will fail due to the cooldown check.
3. Instead, Alice creates a new, empty wallet, Bob's Address.
4. Alice transfers her ZLP tokens from Alice's Address to Bob's Address.
5. Alice then calls `removeLiquidity` from Bob's Address. When the contract checks lastAddedAt[Bob's Address], it finds the value is 0, because Bob's address has never deposited liquidity.

6. The check `0 + cooldownDuration <= block.timestamp` will always pass, allowing the withdrawal to proceed instantly.

This effectively makes the cooldown mechanism completely ineffective for any user who knows how to perform a simple token transfer

Impact

Complete failure of the cooldown feature. Whatever security or economic stability this mechanism was designed to provide is nullified.

Recommendation

Acknowledge that this cooldown is a "weak" control that only affects unsophisticated users. If it is not critical for protocol security, document its limitations and do not rely on it as a robust defense.

Status

Resolved

[M-03] Vault.sol - Unbounded Fees Allow Fund Theft

Description

The `setFees` function in the `Vault.sol` contract allows a privileged address, `gov`, to configure various protocol fees. Crucial validation checks that are meant to limit these fees to a reasonable maximum have been commented out in the code. This omission creates a severe vulnerability where the `gov` account can set fees to arbitrarily high levels, including 100% or more, enabling the direct theft of user assets.

Vulnerability Details

The root cause of the vulnerability lies within the `setFees` function. The code includes several commented-out lines that were intended to validate the fee parameters against predefined maximums.

Vulnerable Code (Vault.sol, lines 337-343):

```
function setFees(
        uint256 _taxBasisPoints,
        uint256 _stableTaxBasisPoints,
        uint256 _mintBurnFeeBasisPoints,
        uint256 _swapFeeBasisPoints,
        uint256 _stableSwapFeeBasisPoints,
        uint256 _marginFeeBasisPoints,
        uint256 _liquidationFeeUsd,
        uint256 _minProfitTime,
        bool _hasDynamicFees
    ) external override {
        _onlyGov();
        // _validate(_taxBasisPoints <= MAX_FEE_BASIS_POINTS, 3);
        // _validate(_stableTaxBasisPoints <= MAX_FEE_BASIS_POINTS, 4);
        // _validate(_mintBurnFeeBasisPoints <= MAX_FEE_BASIS_POINTS, 5);
        // _validate(_swapFeeBasisPoints <= MAX_FEE_BASIS_POINTS, 6);
        // _validate(_stableSwapFeeBasisPoints <= MAX_FEE_BASIS_POINTS, 7);
        // _validate(_marginFeeBasisPoints <= MAX_FEE_BASIS_POINTS, 8);
        // _validate(_liquidationFeeUsd <= MAX_LIQUIDATION_FEE_USD, 9);
        taxBasisPoints = _taxBasisPoints;
        stableTaxBasisPoints = _stableTaxBasisPoints;
        mintBurnFeeBasisPoints = _mintBurnFeeBasisPoints;
```

```
        swapFeeBasisPoints = _swapFeeBasisPoints;
        stableSwapFeeBasisPoints = _stableSwapFeeBasisPoints;
        marginFeeBasisPoints = _marginFeeBasisPoints;
        liquidationFeeUsd = _liquidationFeeUsd;
        minProfitTime = _minProfitTime;
        hasDynamicFees = _hasDynamicFees;
    }
```

Because these `_validate` checks are inactive, the function directly assigns the input values to the state variables that control the fees. For instance, `swapFeeBasisPoints` can be set to `10000` basis points. Since the system's `BASIS_POINTS_DIVISOR` is `10000`, this corresponds to a fee of 100%. Any user performing a swap would have their entire input amount taken as a fee.

Impact

The `gov` can set any of the core operational fees (for swapping, minting/burning USDG, or margin trading) to 100%. The next user to perform that action will have their entire transaction value siphoned into the vault's `feeReserves`. The `gov` can then withdraw these stolen funds using the `withdrawFees` function.

Recommendation

Reinstate the disabled security checks. The commented-out validation lines within the `setFees` function must be uncommented to enforce the intended fee limits.

Status

Resolved

[M-04] PositionRouter.sol - Users Cannot Cancel Orders When Leverage Is Disabled

Description

Users are unable to cancel their own orders when leverage is disabled.

Vulnerability Details

In the event that the admin calls `setIsLeverageEnabled()` and sets `isLeverageEnabled` to false, users are no longer allowed to cancel their own orders due to this check in `_validateExecutionOrCancellation`:

```
if (!isLeverageEnabled && !isKeeperCall) {
    revert("403");
}
```

The users will need a keeper to cancel their order for them, which will result in either the keeper or the user losing the `executionFee`.

Impact

Users lose control of their orders and incur unnecessary execution fee losses.

Recommendation

Allow users to cancel their pending orders in the event that `isLeverageEnabled` is configured to false.

Status

Resolved

Low

[L-01] Contracts - Important functions lack event emissions

Description

In Solidity, events provide a critical logging mechanism that external observers (like dApps, users, or auditors) rely on to track what's happening on-chain. Important functions, such as those that modify contract state, transfer funds, or change ownership, should emit events to provide transparency.

The following critical functions were lacking in event emissions:

```
Router.sol::setGov()

Router.sol::addPlugin()

Router.sol::removePlugin()

Router.sol::approvePlugin()

Router.sol::denyPlugin()

Router.sol::removePlugin()

Router.sol::pluginTransfer()

Router.sol::pluginIncreasePosition()

Router.sol::pluginDecreasePosition()

Router.sol::directPoolDeposit()

Router.sol::increasePosition()

Router.sol::increasePositionETH()

Router.sol::decreasePosition()

Router.sol::decreasePositionETH()

Router.sol::decreasePositionAndSwap()

Router.sol::decreasePositionAndSwapETH()

Router.sol::decreasePositionETH()
```

```
ShortsTracker.sol::setInitData()

Vault.sol::setVaultUtils()

Vault.sol::setErrorController()

Vault.sol::setError()

Vault.sol::setInManagerMode()

Vault.sol::setManager()

Vault.sol::setInPrivateLiquidationMode()

Vault.sol::setLiquidator()

Vault.sol::setIsSwapEnabled()

Vault.sol::setIsLeverageEnabled()

Vault.sol::setMaxGasPrice()

Vault.sol::setGov()

Vault.sol::setPriceFeed()

Vault.sol::setMaxLeverage()

Vault.sol::setBufferAmount()

Vault.sol::setMaxGlobalShortSize()

Vault.sol::setFees()

Vault.sol::setFundingRate()

Vault.sol::setTokenConfig()

Vault.sol::clearTokenConfig()

Vault.sol::withdrawFees()

Vault.sol::addRouter()

Vault.sol::removeRouter()

Vault.sol::setUsdgAmount()

Vault.sol::upgradeVault()

VaultErrorController.sol::setErrors()
```

```
VaultPriceFeed.sol::setGov()

VaultPriceFeed.sol::setChainlinkFlags()

VaultPriceFeed.sol::setAdjustment()

VaultPriceFeed.sol::setUseV2Pricing()

VaultPriceFeed.sol::setIsAmmEnabled()

VaultPriceFeed.sol::setIsSecondaryPriceEnabled()

VaultPriceFeed.sol::setSecondaryPriceFeed()

VaultPriceFeed.sol::setTokens()

VaultPriceFeed.sol::setPairs()

VaultPriceFeed.sol::setSpreadBasisPoints()

VaultPriceFeed.sol::setSpreadThresholdBasisPoints()

VaultPriceFeed.sol::setFavorPrimaryPrice()

VaultPriceFeed.sol::setPriceSampleSpace()

VaultPriceFeed.sol::setMaxStrictPriceDeviation()

VaultPriceFeed.sol::setTokenConfig()

VaultUtils.sol::addPosition()

VaultUtils.sol::removePosition()

ZlpManager.sol::setInPrivateMode()

ZlpManager.sol::setShortsTracker()

ZlpManager.sol::setShortsTrackerAveragePriceWeight()

ZlpManager.sol::setHandler()

ZlpManager.sol::setCooldownDuration()

ZlpManager.sol::setAumAdjustment()
```

Impact

Critical actions (e.g., deposits, stakings) occur silently, making monitoring via block explorers or off-chain tools impossible.

Recommendation

Emit appropriate events in all important state-changing functions.

Status

Resolved

[L-02] Contracts - Implement two two-step ownership transfers for the governance address

Description

The transfer of governance ownership is immediate and lacks verification:

```
function setGov(address _gov) external onlyGov {
        gov = _gov;
    }
```

Once called, ownership is instantly transferred to the specified address without any confirmation from the recipient.

The affected contracts are as follows:

```
BasePositionManager.sol

OrderBook.sol

Router.sol

ShortsTracker.sol

Vault.sol

VaultErrorController.sol

VaultPriceFeed.sol

VaultUtils.sol

ZlpManager.sol
```

Impact

If the new governance's address is entered incorrectly, the contract's control could be lost or handed over to an unintended entity, leading to potential security risks or loss of functionality.

Recommendation

To mitigate this risk, it's advisable to enhance the ownership transfer process by implementing a two-step mechanism.

This two-step process ensures that ownership is only transferred if the new governance explicitly accepts it, thereby preventing accidental transfers to incorrect addresses.

Status

Resolved

[L-03] Vault.sol - Potentially Misleading realisedPnl Output

Description

In the `getPosition` function, the 6th entry in the returned tuple, which represents `hasRealisedProfit`, is true if `position.realisedPnl` is zero. This may be misleading for systems relying on the `getPosition` function, as a position can have been just opened and the `hasRealisedProfit` boolean will be true.

Impact

This issue may mislead dependent systems into incorrectly assuming a new position has realized profit, potentially triggering faulty logic or decisions.

Recommendation

Be sure to clearly document this potentially unexpected behavior.

Status

Resolved

[L-04] ZlpManager.sol - Logical Error In Input Validation Of setShortsTrackerAveragePriceWeight Function

Description

In line 80 of ZlpManager.sol, the function performs the following validation:

```
require(shortsTrackerAveragePriceWeight <= BASIS_POINTS_DIVISOR, "ZlpManager: invalid weight");
```

This validation checks the current value of `shortsTrackerAveragePriceWeight` rather than the new input value `_shortsTrackerAveragePriceWeight`. This means the validation will always pass if the current value is valid, regardless of what the new value is.

Impact

Invalid weight values greater than `BASIS_POINTS_DIVISOR` (10,000) could be set, leading to incorrect pricing calculations for global short positions and potential economic exploits affecting ZLP token pricing.

Recommendation

Change line 80 to validate the input parameter instead of the current state variable.

Status

Resolved

Gas

[G-01] Contracts - Variables that are only set once in the constructor should be declared immutable

Description

In Solidity, variables that are only assigned once within the constructor should be declared as `immutable`, as this provides significant gas savings and improves efficiency. Unlike regular storage variables, which require additional storage slots and incur higher gas costs for both reading and writing, immutable variables are stored directly in the contract's bytecode after being set during deployment.

Affected variables:

```
BasePositionManager.sol::vault, ShortsTracker, router, weth

Router.sol::vault, usdg, weth

ShortsTracker.sol::vault

VaultUtils.sol::vault

ZlpManager.sol::vault, usdg, zlp
```

Recommendation

Variables that are only set in the `constructor` should be declared as `immutable`.

Status

Resolved

QA

[Q-01] Contracts - Floating pragma

Description

The contracts `BasePositionManager.sol`, `OrderBook.sol`, `PositionManager.sol`, `PositionRouter.sol`, and `PositionUtils.sol` use a floating pragma version (^0.6.0). It's crucial to compile and deploy contracts with the same compiler version and settings used during development and testing. Fixing the pragma version prevents compilation with different versions. Using an outdated pragma version could introduce bugs that negatively affect the contract system, while newly released versions may have undiscovered security vulnerabilities.

Recommendation

Change the pragma statement in both contracts to a fixed version, such as pragma solidity 0.6.12. This locks the contract to a specific compiler version.

Status

Resolved

[Q-02] BasePositionManager.sol, VaultUtils.sol - Unused Import

Description

Unused imports increase code complexity and reduce readability, making it easier for subtle bugs or security issues to go unnoticed.

In `BasePositionManager.sol::import "./interfaces/IOrderBook.sol";`, `IOrderBook.sol` is imported but not used anywhere in the contract.

In `VaultUtils.sol`, `Governable.sol` is imported, but the modifier `onlyGov` is not used anywhere in the contract.

Recommendation

Consider removing unused imports to improve code readability.

Status

Resolved

[Q-03] OrderBook.sol - Event Parameter Typo

Description

In the `UpdateSwapOrder` event declaration, the parameter `orderIndex` is misspelled as `ordexIndex`. This can break off-chain services, indexers, and user interfaces that rely on event signatures and parameter names.

Recommendation

Correct the typo from `ordexIndex` to `orderIndex`.

Status

Resolved

Centralisation

Zeus Exchange values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Zeus Exchange seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au